

第九届全国大学生
集成电路创新创业大赛

基于FPGA的 机械臂精确控制系统设计

BY: CICC0906044

目录 Contents

本次杯赛要求选手使用指定的FPGA开发套件控制多自由度机械臂完成将一定数量含有不同形状和颜色图案的正方形物块按照指定规则(赛题图片)进行仓库至目的地的移动、拼接和还原。

01

项目介绍

02

关键技术分析

03

性能指标

04

可优化方向

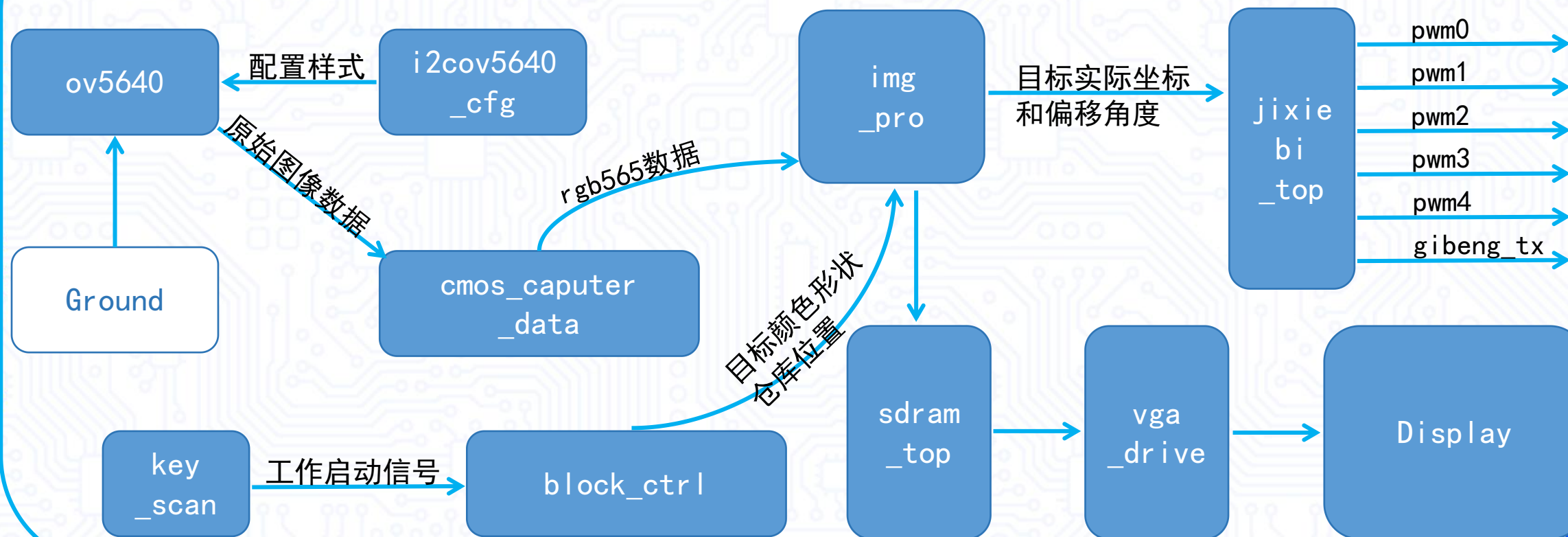


01

项目介绍

Project Introduction

1.1 基于FPGA的机械臂控制系统总流程图



整个工作系统简略分为四个模块，分别是ov5640图像采集，图像处理模块，机械臂控制模块，任务控制模块（用来联系图像处理模块和机械臂处理模块）

01 项目介绍

Project Introduction



1.2 图像采集模块

OV5640是一款高性能500万像素CMOS图像传感器，采用 $1.4\mu\text{m}$ 像素尺寸和OmniBSI技术，支持自动曝光、白平衡等图像控制功能，可输出RGB565或JPEG格式图像。

本设计通过修改正点原子例程，配置其输出 $640\times 480@60\text{Hz}$ RGB565格式。系统采用SDRAM乒乓缓冲机制，将16位256Mb SDRAM划分为双Bank，交替存储和读取图像数据，有效解决高速采集与显示间的速率匹配问题。

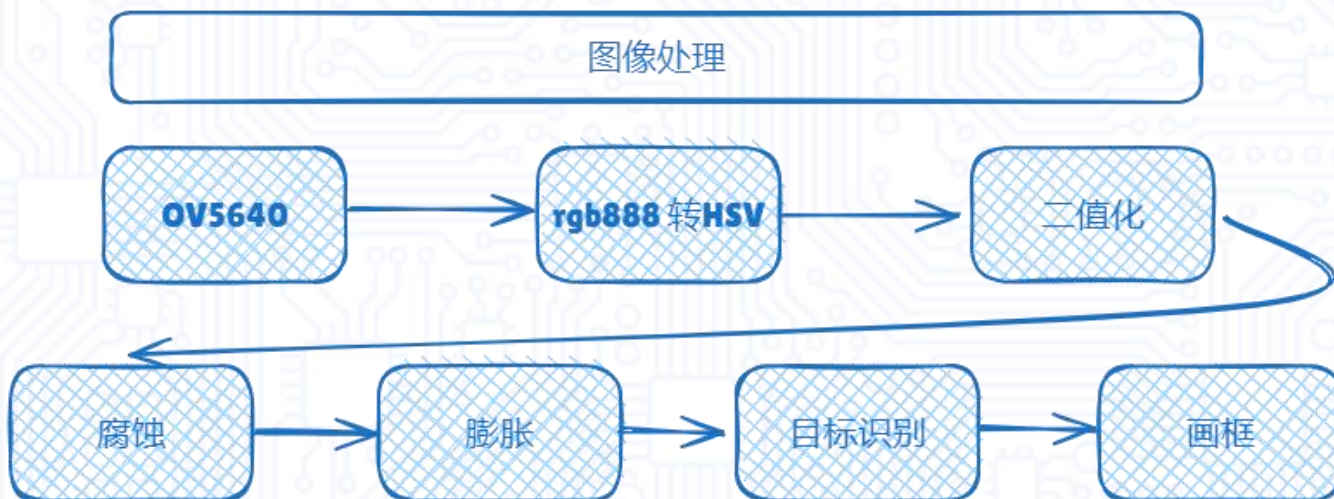
VGA显示部分基于标准25.175MHz时序，将RGB565数据通过数模转换生成模拟信号，配合行场同步信号实现逐行扫描显示。该方案完整实现了从图像采集、缓冲到显示的实时处理流程。



1.3 图像处理模块

在颜色识别任务中，RGB模型的表现往往不如其它颜色模型，例如HSV或者Lab。例如，当光照条件变化时，一个物体的颜色可能会在RGB空间中出现显著的偏移，导致识别困难。这就需要采用更为鲁棒的颜色空间，或者配合其它颜色处理技术来提高颜色识别的准确度和稳定性。

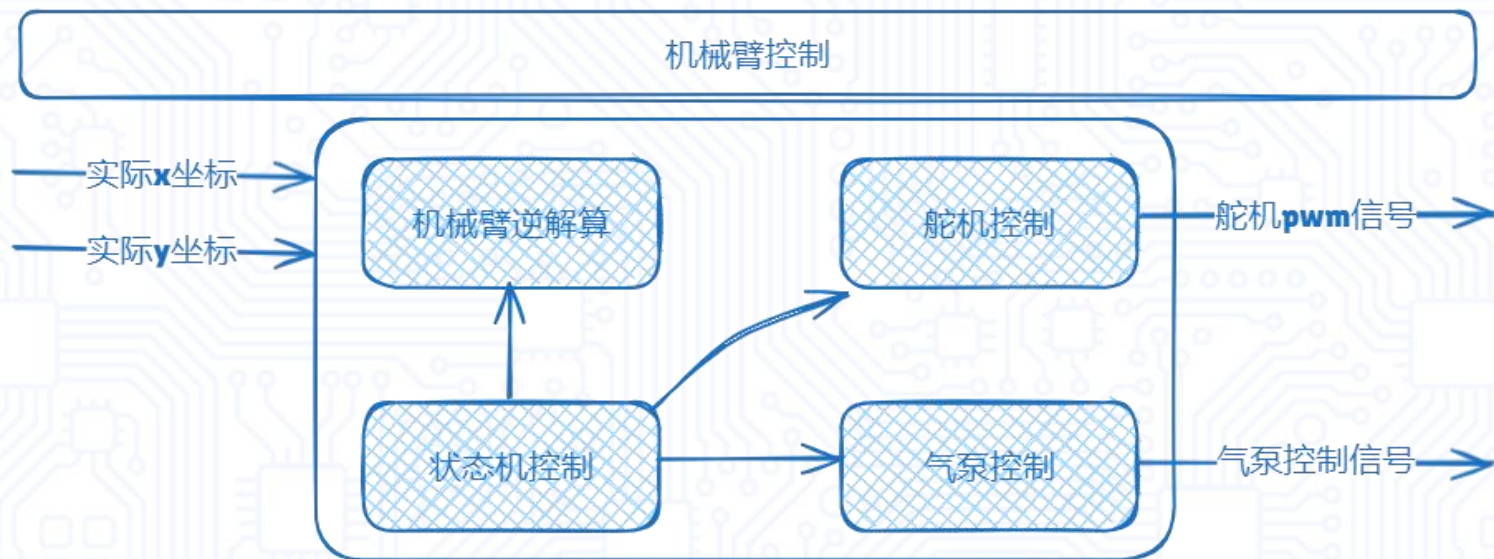
因此，本项目采用先将rgb转换成hsv色彩后进行颜色识别。得到hsv色彩后进行根据目标颜色进行二值化处理得到二值化数据，然后进行形态学滤波，先进行腐蚀操作，再进行膨胀操作，得到更加平滑噪声更少的图像后进行目标识别获得框住目标的四个顶点坐标。



1.4 机械臂控制模块

机械臂控制模块在接受到图像里模块传来的实际坐标后和目标仓库位置后执行动作，一套动作包括逆运动学解算，气泵控制模块，获取目标仓库位置所对应各个关节所运动的关节。

这里由于目标仓库的位置是定死的，为了节省资源将目标仓库的24个槽位设计成查找表的形式，输入为0~23索引地址，输出是对应关节所转动的角度。

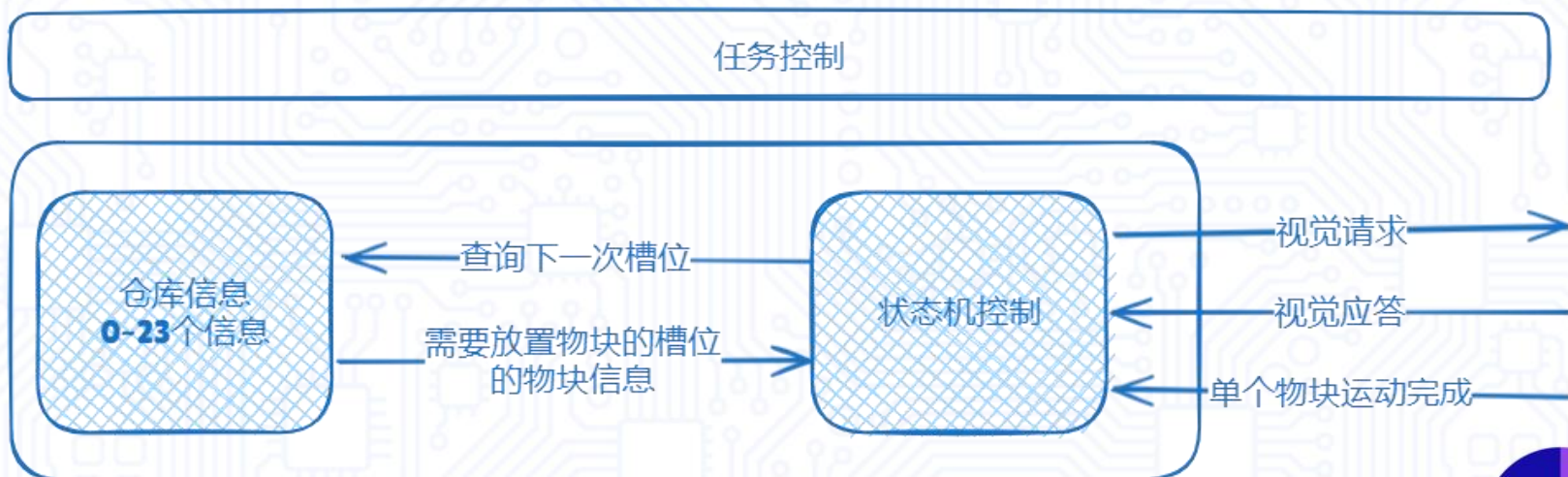


01 项目介绍

Project Introduction

1.5 任务控制模块

该模块用来联系图像信息采集和机械臂运动，使系统正确运作，题目要求是将目标场地的物块使用机械臂吸取将物块移动到仓库的相应位置，我的做法是让仓库来发出请求工作，使用二维数组记录每个槽位的信息（有无物块防止，物块颜色，物块角度），在任务控制模块里查询仓库信息发出视觉请求给图像处理模块，发出目标仓库位置索引给机械臂控制模块，再接受到机械臂控制模块单个物块移动完成的信号后，接着查询下一个需要放置物块的槽位，如此反复，直至到最后一个槽位后，一次工作完成。





02

关键技术分析

Key Technology Analysis

2.1 图像显示

1. 通过SCCB协议配置ov5640摄像头后，OV5640摄像头在RGB565模式下输出 16位数据（R[4:0]+G[5:0]+B[4:0]）。为了节省引脚，只提供 8位数据总线，因此需要分两次传输（先高8位，后低8位）。摄像头输出八位rgb数据，我们需要接受俩次数据进行拼接得到一个完整的rgb565数据。

```
//8位数据转16位RGB565数据
```

```
always @(posedge cam_pclk or negedge rst_n) begin
    if(!rst_n) begin
        cmos_data_t <= 16'd0;
        cam_data_d0 <= 8'd0;
        byte_flag <= 1'b0;
    end
    else if(cam_href) begin
        byte_flag <= ~byte_flag;
        cam_data_d0 <= cam_data;
        if(byte_flag)
            cmos_data_t <= {cam_data_d0, cam_data};
        else;
    end
    else begin
        byte_flag <= 1'b0;
        cam_data_d0 <= 8'b0;
    end
end
```

2.1 图像显示

2. 由于摄像头持续采集的图像数据速率（约14.7MB/s）与VGA显示读取的速率需要精确匹配，系统采用了先进的乒乓缓存机制。该机制将SDRAM物理存储空间划分为两个逻辑存储区（Bank A和Bank B），通过巧妙的状态机控制实现读写操作的交替进行。具体工作流程是：当摄像头数据正在写入Bank A时，VGA控制器从Bank B读取数据；当一帧图像采集完成后，立即切换为摄像头数据写入Bank B，同时VGA从Bank A读取。这种双缓冲结构完全避免了读写冲突，确保了数据的完整性和显示的流畅性。采用的正点原子历程实现乒乓操作的关键代码。

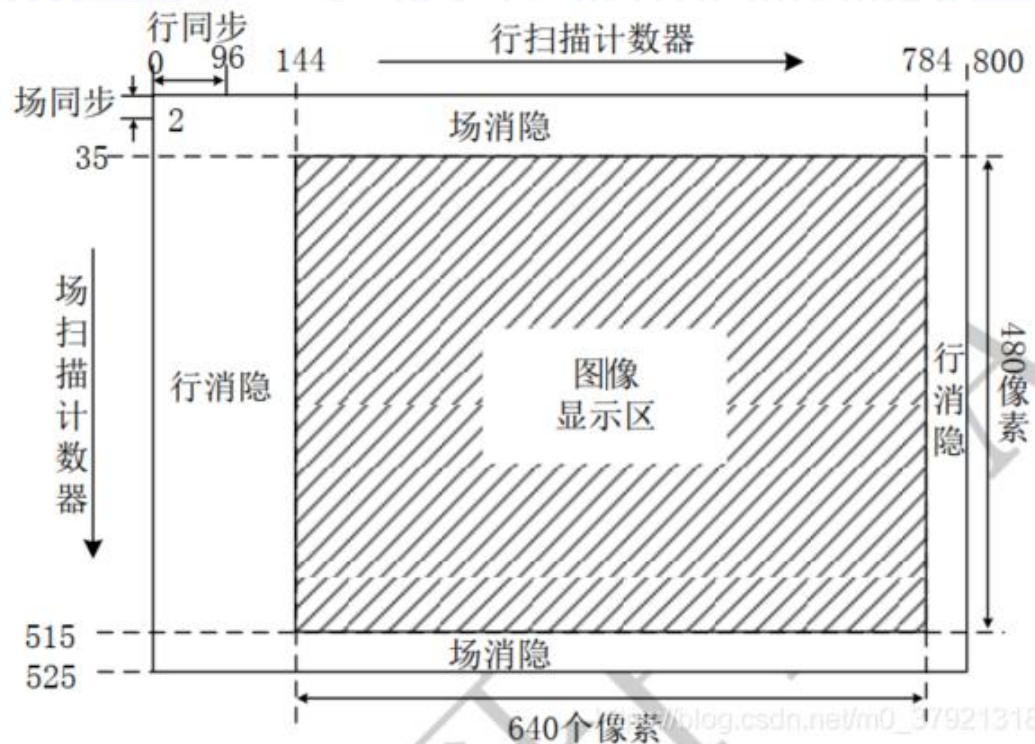
```
if(sdram_pingpang_en) begin
    if(sdram_wr_addr[22:0] < wr_max_addr - wr_length)
        sdram_wr_addr <= sdram_wr_addr + wr_length;
    else begin
        rw_bank_flag <= ~rw_bank_flag;
        sw_bank_en <= 1'b1;
    end
end
end

//若未到达写SDRAM的结束地址，则写地址+1
//SDRAM 读写乒乓使能
//切换BANK
//拉高切换BANK使能信号

//若突发写SDRAM结束，更改写地址
```

2.1 图像显示

3. 在显示输出端，VGA控制器需要精确生成符合VGA标准的时序信号。对于 $640 \times 480@60\text{Hz}$ 的分辨率，系统使用 25.175MHz 的像素时钟（由锁相环ip核生成），每个完整的行周期包含800个时钟周期，其中640个用于有效像素显示，160个为行消隐期；每个完整的场周期包含525行，其中480行为有效显示行，45行为场消隐期。控制器从SDRAM中读取的RGB565数据需要通过简单的电阻网络实现数模转换，生成模拟的VGA信号。同时，严格按标准生成的HSYNC和VSYNC同步信号确保了显示设备的正确扫描。

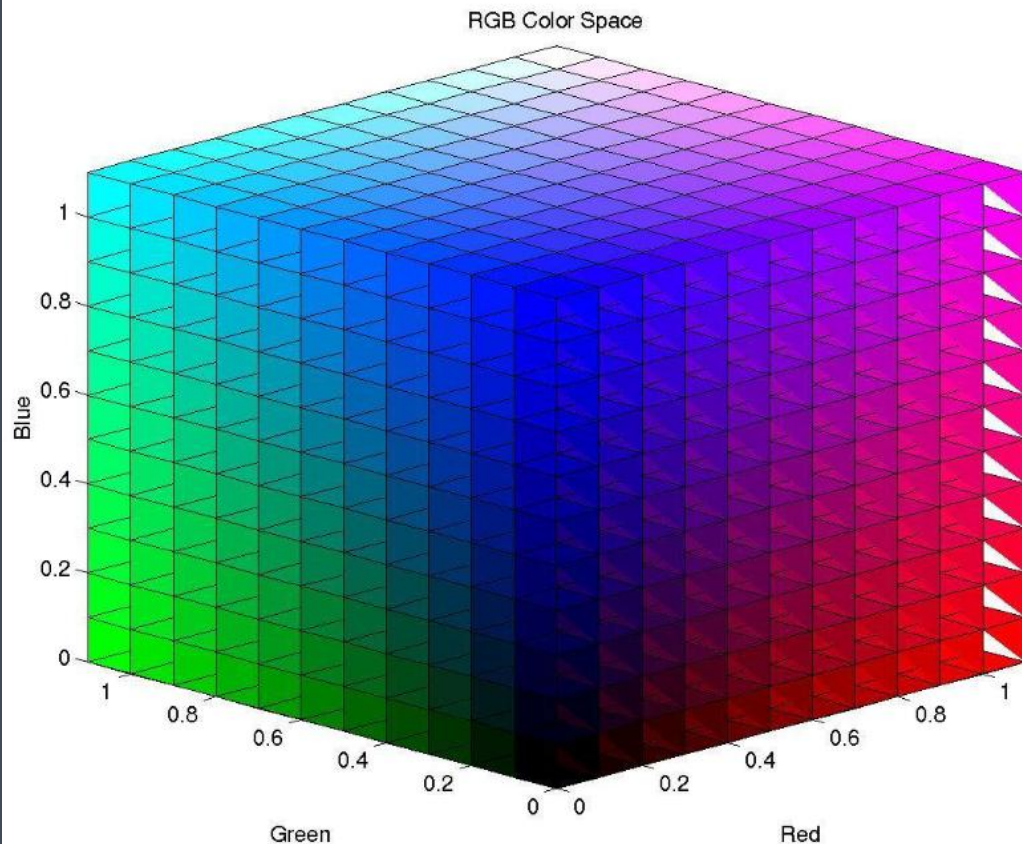


02 关键技术分析

Key Technology Analysis

另外为方便图像坐标系和机械臂坐标系的转换，在显示器的中心位置我们增加了一个十字，与场地中心进行匹配。





2.2 图像处理

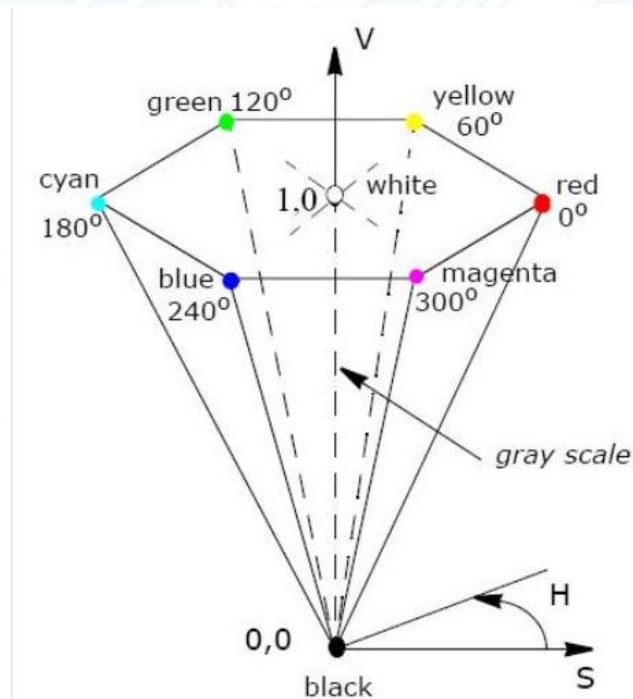
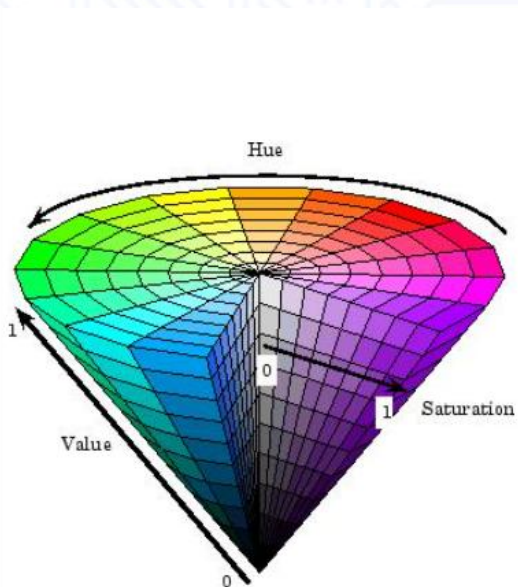
2.2.1 RGB色彩模型

RGB色彩空间使用Red红色、Green绿色、Blue蓝色三个分量来表示颜色，每个分量的范围是 $[0\%, 100\%]$ 。在计算机中，大部分使用RGB色彩空间的程序都使用1字节来存储其颜色分量，这时每个分量的范围就是我们常见的 $[0, 255]$ 了，本文后续部分也会使用这个RGB范围。

在RGB色彩空间下，颜色的色度，亮度，饱和度等信息无法区分，这在处理中是相当麻烦的，对于本项目，需要处理的是颜色的色度分量，而在RGB色彩空间下不好将不同色度区分开，因此引入HSV色彩空间模型。

02 关键技术分析

Key Technology Analysis



2.2.2 HSV色彩模型

HSV (Hue, Saturation, Value) 是根据颜色的直观特性由A. R. Smith在1978年创建的一种颜色空间，也称六角锥体模型(Hexcone Model)。在HSV模型中，颜色是由色度 (Hue)，饱和度 (Saturation)，明度 (Value) 共同组成。



02 关键技术分析

Key Technology Analysis



1. 明度 (Value) 计算

$$V = \max(R, G, B) \in [0, 255]$$

2. 饱和度 (Saturation) 计算

$$S = \begin{cases} 0 & \text{if } V = 0 \\ 255 \times \frac{V - \min(R, G, B)}{V} & \text{otherwise} \end{cases} \in [0, 255]$$

3. 色相 (Hue) 计算 (分段函数)

$$H = \begin{cases} 60^\circ \times \frac{G-B}{\Delta} & \text{if } V = R \\ 60^\circ \times \left(2 + \frac{B-R}{\Delta}\right) & \text{if } V = G \\ 60^\circ \times \left(4 + \frac{R-G}{\Delta}\right) & \text{if } V = B \end{cases} \in [0^\circ, 360^\circ]$$

其中:

$$\Delta = V - \min(R, G, B)$$

在HSV模型下，通过H分量即色度即可区分不同的颜色，从而通过设定不同的颜色阈值范围，达到区分颜色的目的，如下将介绍本模块中使用的RGB转HSV后通过改变H分量，再将改变H值后的HSV转化为RGB，从而实现将一种颜色转化为另一种颜色。

本项目中将红色和黄色转变为蓝色，再通过灰度模块提取蓝色色度分量，从而实现提取不同的颜色的目的，并且在本模块就区分出了四种颜色，在后续处理模块只需要区分出形状即可。另外为了后续方便同一参数位宽，我们将h, s, v的数值都映射在0~255的区间上，只需6个八位宽数据即可表示阈值。



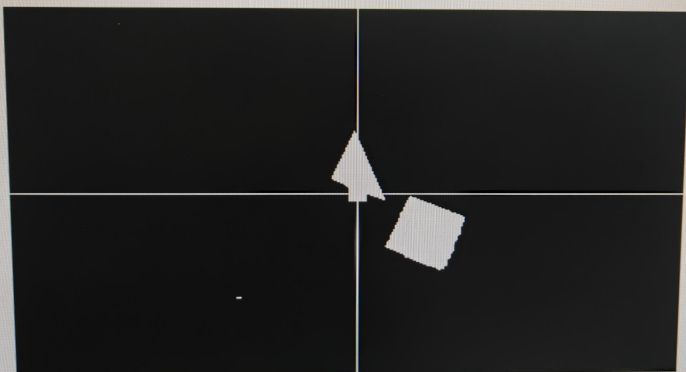
02 关键技术分析

Key Technology Analysis

2.2.3 二值化

在得到hsv数据，只需对图像数据进行二值化即可的到二值化数据利于接下来的图像处理。

通过比较阈值，在参数范围内的像素点赋值为白色，其他则是黑色。二值化后的图像如下：



```
// 颜色定义参数
localparam COLOR_RED    = 2'b00;
localparam COLOR_BLUE   = 2'b01;
localparam COLOR_YELLOW = 2'b10;
localparam COLOR_BLACK  = 2'b11;

// 红色阈值 (注意红色在Hsv色轮的两端)
localparam RED_H_MIN_LOW  = 8'd0;
localparam RED_H_MAX_LOW  = 8'd10;
localparam RED_H_MIN_HIGH = 8'd160;
localparam RED_H_MAX_HIGH = 8'd180;
localparam RED_S_MIN      = 8'd100;
localparam RED_S_MAX      = 8'd255;
localparam RED_V_MIN      = 8'd50;
localparam RED_V_MAX      = 8'd255;

// 蓝色阈值
localparam BLUE_H_MIN     = 8'd150;
localparam BLUE_H_MAX     = 8'd175;
localparam BLUE_S_MIN     = 8'd50;
localparam BLUE_S_MAX     = 8'd255;
localparam BLUE_V_MIN     = 8'd0;
localparam BLUE_V_MAX     = 8'd255;

// 黄色阈值
localparam YELLOW_H_MIN   = 8'd25;
localparam YELLOW_H_MAX   = 8'd35;
localparam YELLOW_S_MIN   = 8'd100;
localparam YELLOW_S_MAX   = 8'd255;
localparam YELLOW_V_MIN   = 8'd50;
localparam YELLOW_V_MAX   = 8'd255;

// 黑色阈值 (主要看v值)
localparam BLACK_H_MIN    = 8'd0;
localparam BLACK_H_MAX    = 8'd255;
localparam BLACK_S_MIN    = 8'd0;
localparam BLACK_S_MAX    = 8'd255;
localparam BLACK_V_MIN    = 8'd0;
localparam BLACK_V_MAX    = 8'd50;

// 中间信号
```

02 关键技术分析

Key Technology Analysis



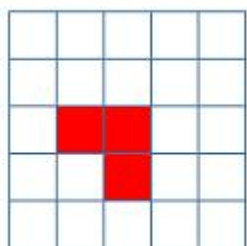
原始图像



开运算图像

2.2.4 形态学滤波-开操作

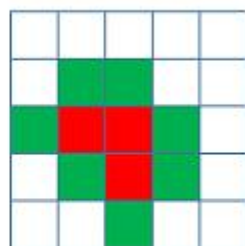
图像开运算是图像依次经过腐蚀、膨胀处理后的过程。图像被腐蚀后，去除了噪声，但是也压缩了图像；接着对腐蚀过的图像进行膨胀处理，可以去除噪声，并保留原有图像。如图所示



结构A



结构B



结构A膨胀后

2. 膨胀

结构A被结构B膨胀的定义为，

$$A \oplus B = \{z \mid (B^{\wedge})_z \cap A \neq \emptyset\}$$

可以理解为，将结构B在结构A上进行卷积操作，如果移动结构B的过程中，与结构A存在重叠区域，则记录该位置，所有移动结构B与结构A存在交集的位置的集合为结构A在结构B作用下的膨胀结果。

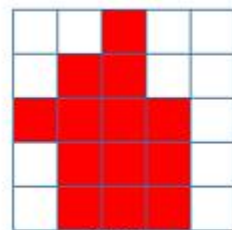
图示中红色框内的区域表示结构A在结构B的作用下膨胀的结果。

1. 腐蚀

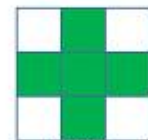
结构A被结构B腐蚀的定义为，

$$A \odot B = \{z \mid (B)_z \subseteq A\}$$

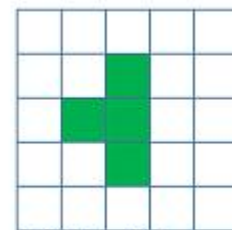
可以理解为，移动结构B，如果结构B与结构A的交集完全属于结构A的区域内，则保存该位置点，所有满足条件的点构成结构A被结构B腐蚀的结果。



结构A



结构B

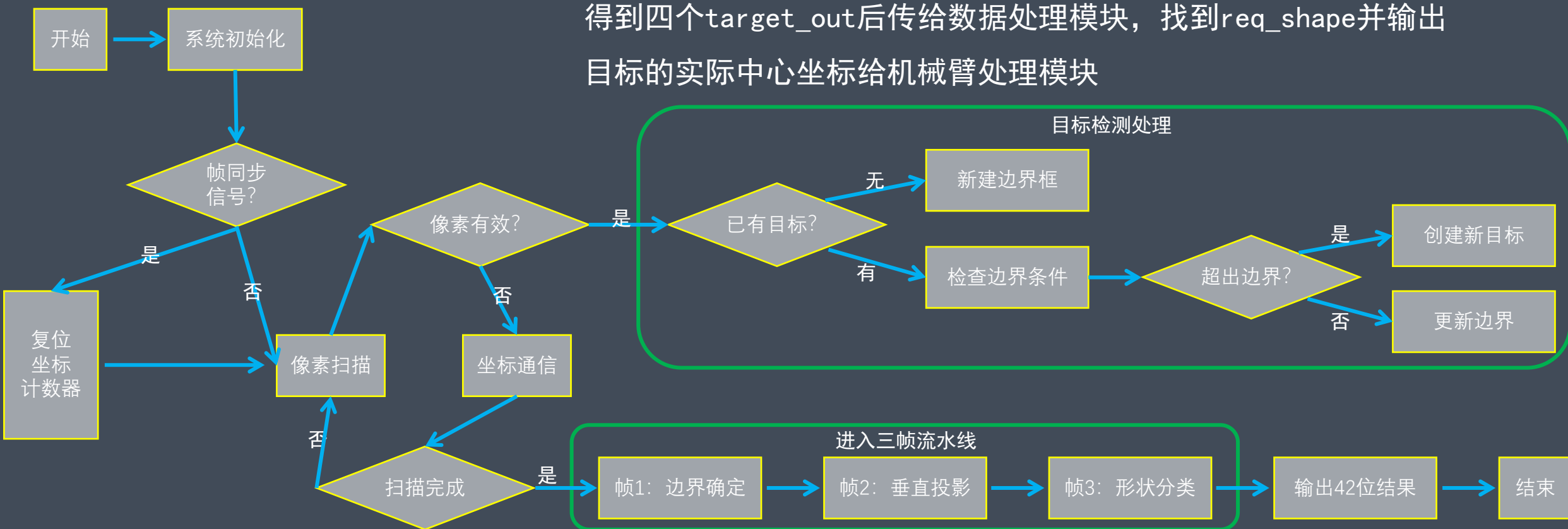


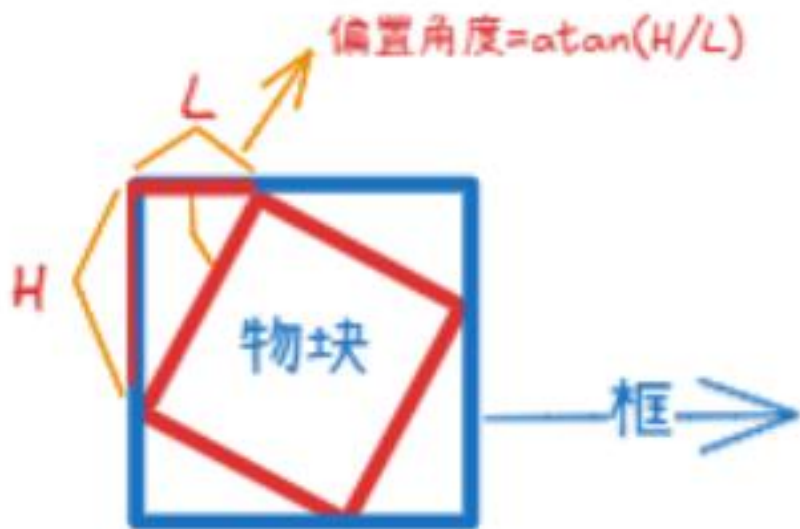
结构A腐蚀后

2.2.5 目标识别

目标识别参考b站大磊老师识别纸钞的思路，目标检测逻辑采用动态边界框扩展算法，通过逐像素扫描和状态机控制实现实时多目标跟踪：系统在每帧开始时初始化所有目标边界框，当检测到有效像素（per_img_Bit=1）时，首先检查该像素坐标是否超出所有现有目标的扩展边界区域（边界外扩MIN_DIST像素），若超出则创建新目标（如检测到(100, 80)像素时建立初始边界框）；对于已关联的目标，系统动态更新其边界框范围（如当发现更靠右的(150, 120)像素时，将右边界从100扩展到150），同时通过三帧流水线机制确保稳定的目标跟踪——首帧完成目标发现和边界确定，中间帧进行垂直投影面积计算，末帧执行形状分类和结果输出。通过(x_cnt, y_cnt)坐标计数器实现扫描同步，利用target_pos寄存器组维护各目标的实时边界信息（包含40位边界坐标和1位有效标志），并采用vsync_cnt状态机控制三阶段处理流程，最终输出包含形状编码、边界坐标和面积特征的42位结构化数据。

得到四个target_out后传给数据处理模块，找到req_shape并输出目标的实际中心坐标给机械臂处理模块





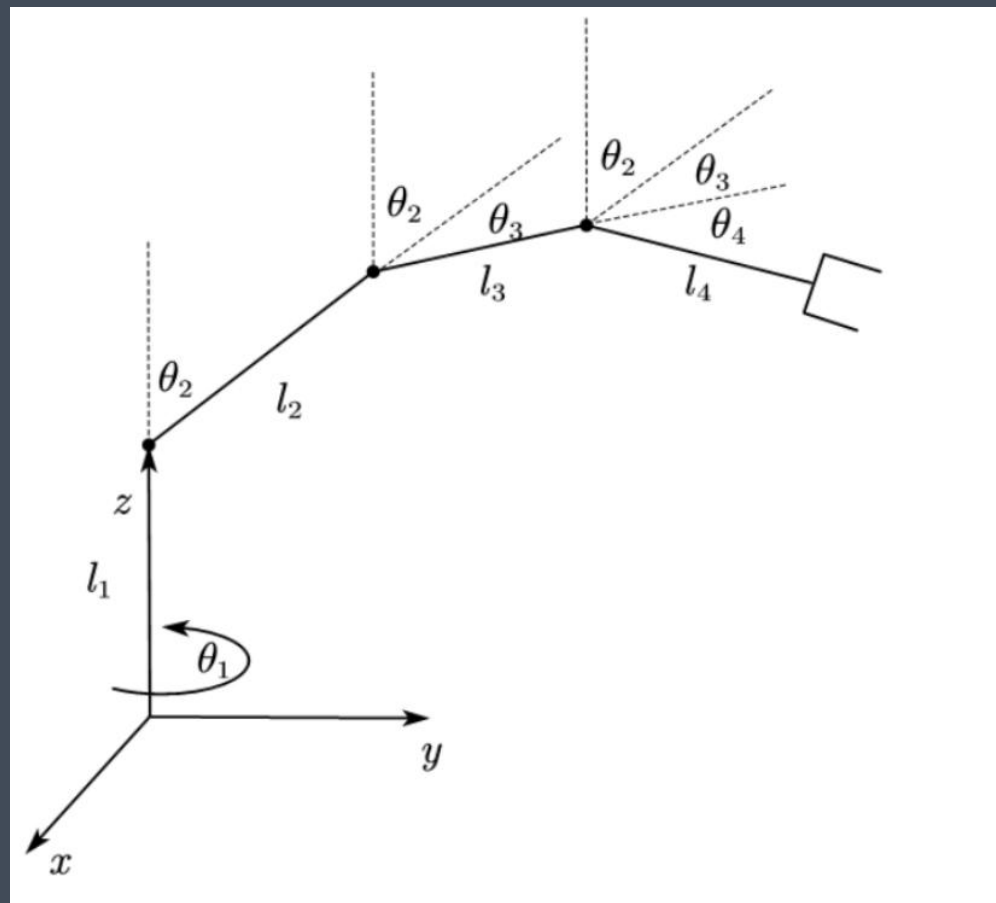
2.2.6 角度信息获取与坐标转换

关于正方形角度信息获取，在得到正方形物块框后，以框的左上角为七点分别向左和向下延伸，再碰到正方形物块，也就是碰到有效像素的时候停止延伸，记录下向左和向下延伸的长度，计算比值后进行atan计算即可得到正方形的偏置角度。

2.3 机械臂控制

2.3.1 逆运动学解算

得到实际坐标后，需要进行逆运动学解算，因为机械臂吸取时末端保持与水平面垂直效果较好，这为机械臂几何逆解提供了方便。





02 关键技术分析

Key Technology Analysis



由于末端与水平面垂直，有条件角度2+角度2+角度4的和为90度，运动模型变为右图：

利用三角性质，得到下面的表达式。

计算过程如下：

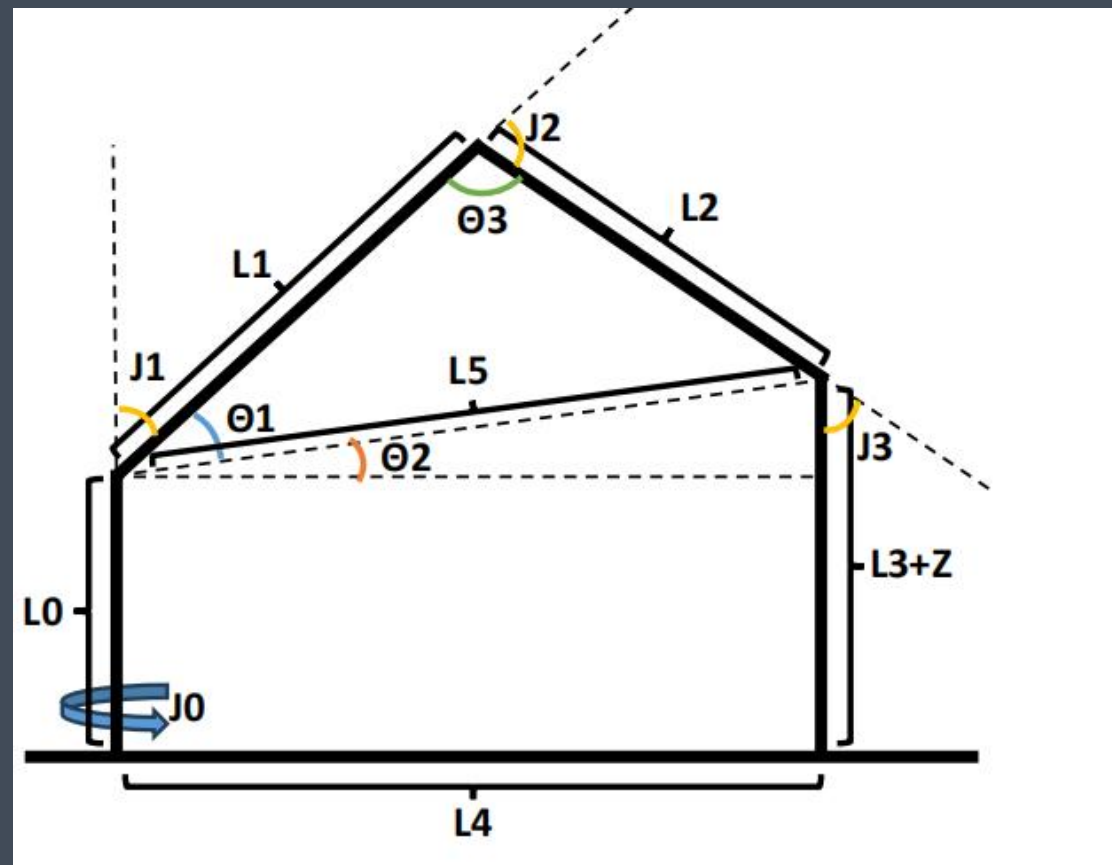
$$L_4 = \sqrt{x^2 + y^2}, L_5 = \sqrt{(L_3 + z - L_0) + L_4^2}$$

$$\cos\theta_1 = \frac{L_1^2 + L_5^2 - L_2^2}{2L_1L_5}, \cos\theta_3 = \frac{L_1^2 + L_2^2 - L_5^2}{2L_1L_2}$$

$$\theta_1 = \arccos(\cos\theta_1), \theta_2 = \arccos\left(\frac{L_4}{L_5}\right), \theta_3 = \arccos(\cos\theta_3),$$

$$j_0 = \arccos\left(\frac{y}{L_4}\right), j_1 = 90^\circ - \theta_1 - \theta_2,$$

$$j_2 = 180^\circ - \theta_3, j_3 = \theta_1 + \theta_2 + \theta_3 - 90^\circ$$





整个模块只需要使用两个IP核：DIV和sqrt。其中arccos与atan的计算使用查找表实现
逆解需要1584个逻辑单元

▽ Inverse_kinematics_ak:u_ik	1584 (514)	660 (430)	0 (0)	24576	3
> Acos_Rom:Acos_Rom	0 (0)	0 (0)	0 (0)	24576	3
> DIV:DIV	812 (0)	200 (0)	0 (0)	0	0
> lpm_mult:Mult0	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult1	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult2	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult3	0 (0)	0 (0)	0 (0)	0	0
> lpm_mult:Mult4	0 (0)	0 (0)	0 (0)	0	0





02 关键技术分析

Key Technology Analysis



2.3.2 仓库位置机械臂关节对应角度的存储

为了节省计算量，再机械臂吸取到物块后，我们采用将目标仓库位置机械臂关节对应角度采用查表表的形式存储，24个槽位，每个槽位对应的角度有四个，每个角度（扩大十倍）使用11位宽有符号数储存。

只需输入目标仓库的槽位的索引即得到相应的旋转角度。

```

-- 机械臂控制ROM数据（严格12位有符号数版本）
-- 格式：{J3[11:0], J2[11:0], J1[11:0], J0[11:0]}
-- 每行12个十六进制字符（无分隔符），表示48位数据
WIDTH = 48;
DEPTH = 24;
ADDRESS_RADIX = DEC;
DATA_RADIX = HEX;

CONTENT BEGIN
-- Y=0
0 : ABC05742228F; --j0: -1348 (0xABC) j1: 86 (0x057) j2: 1057 (0x422) j3: 655 (0x28F)
1 : B210CB38C2B1; --j0: -1247 (0xB21) j1: 202 (0x0CB) j2: 907 (0x38C) j3: 689 (0x2B1)
2 : B651522BF2F7; --j0: -1178 (0xB65) j1: 337 (0x152) j2: 703 (0x2BF) j3: 759 (0x2F7)
3 : B9620618537D; --j0: -1130 (0xB96) j1: 518 (0x206) j2: 388 (0x185) j3: 892 (0x37D)

-- Y=1
4 : B4BFEE492288; --j0: -1205 (0xB4B) j1: -17 (0xFEE) j2: 1169 (0x492) j3: 647 (0x288)
5 : B9F07C3F5297; --j0: -1120 (0xB9F) j1: 124 (0x07C) j2: 1012 (0x3F5) j3: 663 (0x297)
6 : BD210D32B2D0; --j0: -1069 (0xBD2) j1: 269 (0x10D) j2: 810 (0x32B) j3: 719 (0x2D0)
7 : BF41B621633C; --j0: -1035 (0xBF4) j1: 438 (0x1B6) j2: 533 (0x216) j3: 828 (0x33C)

-- Y=2
8 : C1AFAB4CE28F; --j0: -998 (0xC1A) j1: -84 (0xFAB) j2: 1229 (0x4CE) j3: 654 (0x28F)
9 : C3D05042A28E; --j0: -963 (0xC3D) j1: 79 (0x050) j2: 1065 (0x42A) j3: 654 (0x28E)
10 : C500EA35F2BF; --j0: -944 (0xC50) j1: 234 (0x0EA) j2: 862 (0x35F) j3: 702 (0x2BF)
11 : C5C192254322; --j0: -932 (0xC5C) j1: 402 (0x192) j2: 595 (0x254) j3: 801 (0x322)

-- Y=3
12 : D06FAB4CE28F; --j0: -761 (0xD06) j1: -84 (0xFAB) j2: 1229 (0x4CE) j3: 654 (0x28F)
13 : CE305042A28E; --j0: -796 (0xCE3) j1: 79 (0x050) j2: 1065 (0x42A) j3: 654 (0x28E)
14 : CD00EA35F2BF; --j0: -815 (0xCD0) j1: 234 (0x0EA) j2: 862 (0x35F) j3: 702 (0x2BF)
15 : CC4192254322; --j0: -827 (0xCC4) j1: 402 (0x192) j2: 595 (0x254) j3: 801 (0x322)

-- Y=4
16 : DD5FEE492288; --j0: -554 (0xDD5) j1: -17 (0xFEE) j2: 1169 (0x492) j3: 647 (0x288)
17 : D8107C3F5297; --j0: -639 (0xD81) j1: 124 (0x07C) j2: 1012 (0x3F5) j3: 663 (0x297)
18 : D4E10D32B2D0; --j0: -690 (0xD4E) j1: 269 (0x10D) j2: 810 (0x32B) j3: 719 (0x2D0)
19 : D2C1B621633C; --j0: -724 (0xD2C) j1: 438 (0x1B6) j2: 533 (0x216) j3: 828 (0x33C)

-- Y=5
20 : E6405742228F; --j0: -411 (0xE64) j1: 86 (0x057) j2: 1057 (0x422) j3: 655 (0x28F)
21 : DFF0CB38C2B1; --j0: -512 (0xDFF) j1: 202 (0x0CB) j2: 907 (0x38C) j3: 689 (0x2B1)
22 : DBB1522BF2F7; --j0: -581 (0xDBB) j1: 337 (0x152) j2: 703 (0x2BF) j3: 759 (0x2F7)
23 : D8A20618537D; --j0: -629 (0xD8A) j1: 518 (0x206) j2: 388 (0x185) j3: 892 (0x37D)

END;
```





2.3.3 抓取误差设计

对于误差设计，总的来说影响机械臂吸取误差的因素有：

摄像头图像获取实际坐标误差，我们用的ov5640摄像头有些许畸变，但在我实际测试坐标是发现图像处理得到的坐标和实际坐标误差在0.5ms以内。这是一个可以接受并且影响不大的误差。

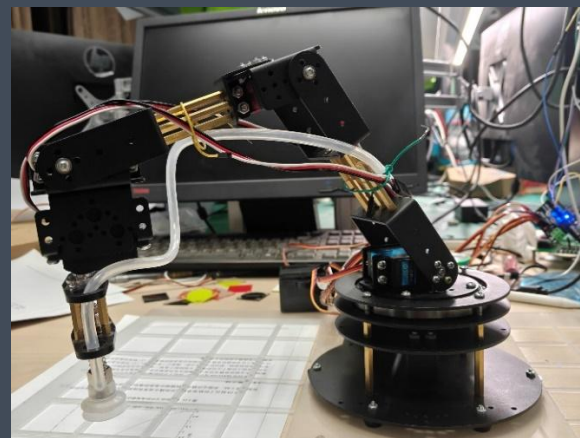
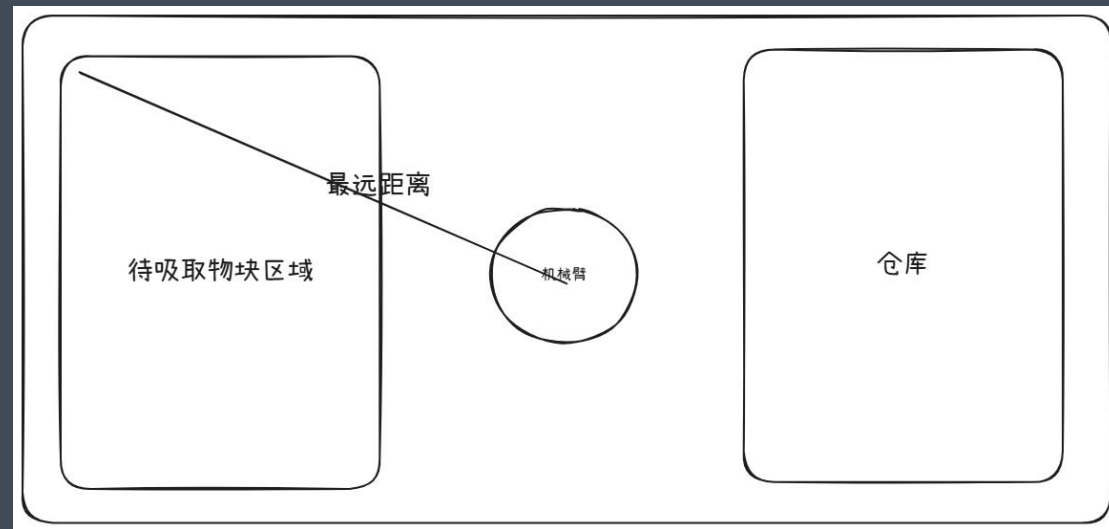
逆解误差，使用Verilog语言实现的逆解存在开平方误差，反三角函数误差，除法误差，但实际我们编写仿真文件测试时候发现误差并不大，将使用Verilog语言解算出来的结果与c语言解算的结果相比较，发现误差也是最大1度左右，对于我们使用理论的精度为0.7度的舵机来说也是一个可以接受的范围

坐标 (x, y, z) 单位 (mm)	verilog逆解(0.1°)	c语言逆解(1°)	平均误差(1°)
(112, 240, 40)	(649, 512, 401, 887)	(65, 52, 39, 90)	0.575
(112, 195, 40)	(600, 336, 716, 748)	(60, 34, 70, 76)	0.8
(112, 150, 40)	(532, 203, 910, 687)	(53, 20, 91, 69)	0.2
(112, 105, 40)	(429, 86, 1063, 651)	(43, 9, 106, 66)	0.425
(76, 105, 40)	(573, -17, 1175, 642)	(57, -2, 117, 65)	0.475



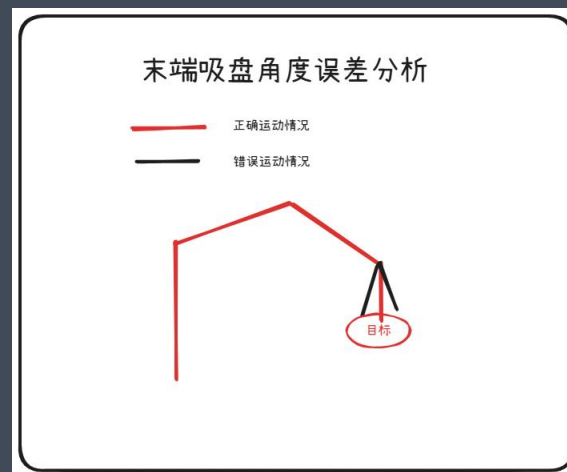
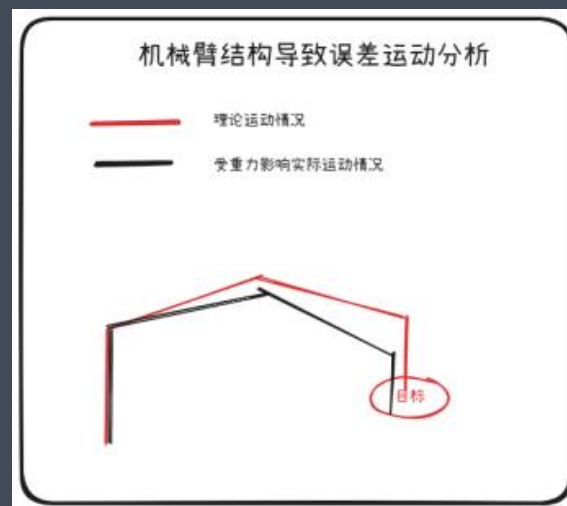
舵机误差与机械结构，舵机误差的组成成分有两部分，一是舵机本身转动的误差，我们所购置的舵机官方说是0.7度左右误差。具体和舵机本身机械性能有关。其次就是误差的主要成分，因为机械臂物理结构所导致的舵机因为重力产生的误差。由于本次地图较大如果将机械臂放置在地图的中心，理论上地图上需要吸取物块离机械臂中心的距离在265mm左右。

我们在淘宝使用所购置的机械臂支架组装后并不能达到最远距离，因此我采用在关节链接处使用同轴连接达到延长机械臂长度的效果



由于加长的大臂与小臂的长度，随之而来就是导致整个机械臂因为重力导致大臂关节与小臂关节会不受舵机角度控制的下降一段角度，会导致在机械臂运动到指定位置后距离目标物块还存在一段距离不够。

但也是由于机械臂的大臂和小臂加长，使得在运动到不同距离的时候，大臂和小臂的误差角度的动态性减少，几乎都是最大误差角度，我们实测发现大臂误差角度为5度左右，小臂关节误差角度在3度左右。补偿大臂和小臂误差后会使得末端关节的水平距离与目标物块一致，但是末端吸盘的角度与目标物块的中心位置可能有误差，如下图所示





02 关键技术分析

Key Technology Analysis



整个误差需要根据距离动态补偿代码实现如下：

```
// 动态调整补偿值（根据距离）
always@(posedge Clk or negedge Rst_n) begin
    if(!Rst_n)
        adjust2 <= 'd0;
    else if(cnt_s == 'd4) begin
        if(L4 >= 230)
            adjust2 <= 'd0;
        else if(L4>=185)
            adjust2 <= 'd30;
        else if(L4>=150)
            adjust2 <= 'd30;
        else adjust2 <= 'd60;
    end
end
```

逆解最终角度输出：

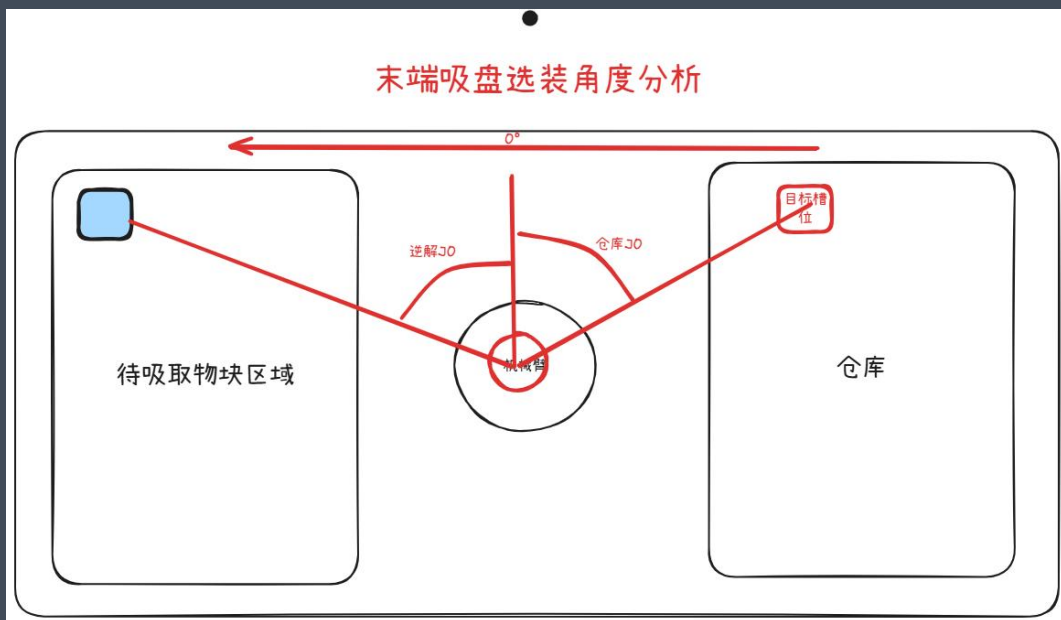
```
end
else if(valid_a_r[2]) begin
    J0 <= J0r+20;
    J1 <= J1r-30;
    J2 <= J2r-50;//50
    J3 <= J3r+adjust2; // 应用动态调整值
end
end
```



2.3.4 角度偏移处理

对于末端吸盘转动角度设计，在得到偏置角度后，将吸取物块旋转至目标仓库放置的角度后，还需考虑由于机械臂地盘旋转导致的角度偏差，图解如下：

可知末端选装角度为逆解J0减去仓库J0的相反数
代码实现如下：



```
always @(*) begin
    // 1. 计算原始角度差 (自动处理溢出)
    j4_angle_raw = target_j0 - store_j0;

    // 2. 将角度差映射到[0, 360)范围
    if ($signed(j4_angle_raw) < 0)
        j4_angle_raw = j4_angle_raw + ANGLE_360;
    else if ($signed(j4_angle_raw) >= ANGLE_360)
        j4_angle_raw = j4_angle_raw - ANGLE_360;

    // 3. 限制到[0, 180]范围
    if ($signed(j4_angle_raw) > ANGLE_180)
        j4_angle_raw = ANGLE_360 - j4_angle_raw; // 取镜像角度

    // 4. 取负数得到[-180, 0]范围
    j4_angle_final = -j4_angle_raw;
end
```



2.3.5 单个吸取状态机设计

对于单个目标吸取，我们设计的是先让机械臂运行到目标物块的上方4厘米处停留1s等待机械臂稳定后再运行到2厘米的位置并开启停留1s在运动到高度0厘米位置进行吸取3s。

在实际测试过程，我们发现机械臂在完成一次运动从仓库位置运动到下一个目标位置的过程中，由于各个机械臂并行运动，如果末端反应相较次末端或者大臂关节速度较慢，可能会出现机械臂碰到地面的情况，为了避免这种情况我们测试得出结论目标准备抓取准备位置的高度和运动到仓库存储位置上方高度为4cm的时候，不会出现机械臂碰地的情况，另外机械臂各个关节的速度也有影响，末端关节的运动速度应该是最快。向下关节运动速度依次减少。这样设计避免复杂的运动轨迹规划，实现使用较为简单的轨迹控制实现本项目的正确吸取任务。



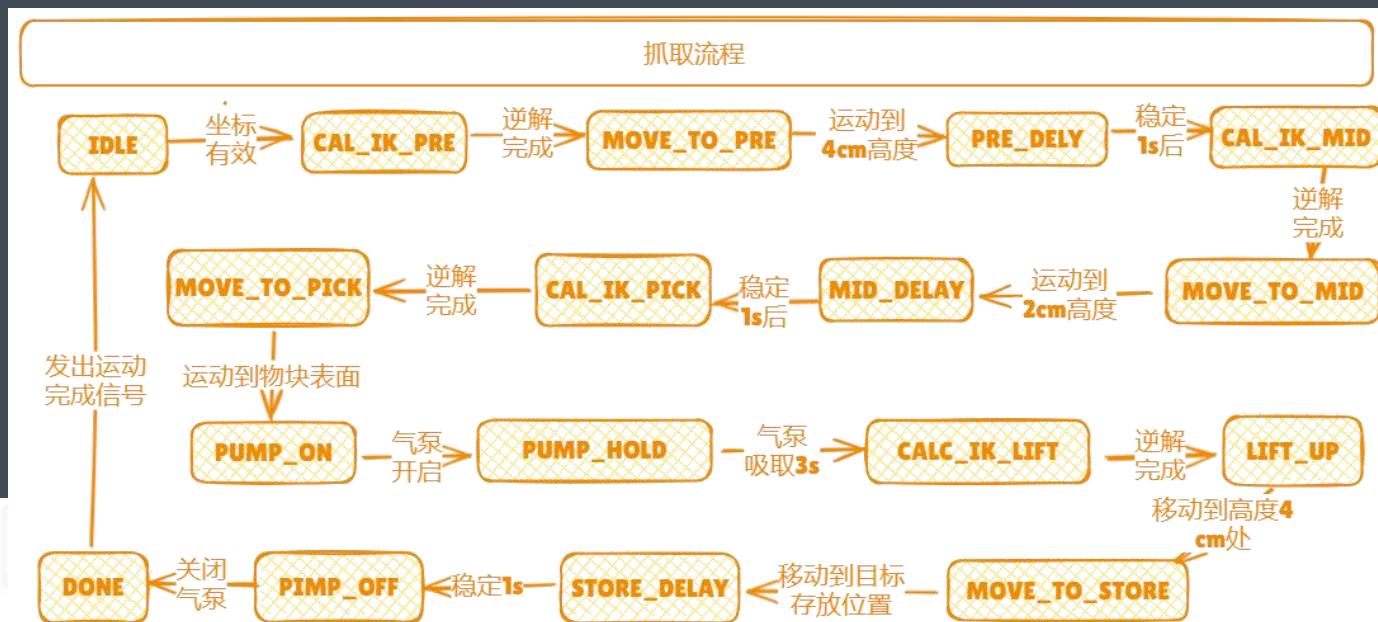
02 关键技术分析

Key Technology Analysis



在实际测试过程，我们发现机械臂在完成一次运动从仓库位置运动到下一个目标位置的过程中，由于各个机械臂并行运动，如果末端反应相较次末端或者大臂关节速度较慢，可能会出现机械臂碰到地面的情况。

为了避免这钟情况我们测试得出结论目标准备抓取准备位置的高度和运动到仓库存储位置上方高度为4cm的时候，不会出现机械臂碰地的情况，另外机械臂各个关节的速度也有影响，末端关节的的运动速度应该是最快。向下关节运动速度依次减少。这样设计避免复杂的运动轨迹规划，实现使用较为简单的轨迹控制实现本项目的正确吸取任务。



02 关键技术分析

Key Technology Analysis

2. 4任务控制

2. 4. 1 目标位置信息传递

为了节省目标仓库的4息储存使用rom查找表实现，每个槽位的信息包括：位置索引，颜色，形状，放置角度。可以使用一个7位宽，深度为24的rom实现：

DEPTH = 24; -- 24个存储单元
WIDTH = 7; -- 每个单元7位宽
ADDRESS_RADIX = DEC; -- 地址采用十进制表示
DATA_RADIX = BIN; -- 数据采用二进制表示

CONTENT
BEGIN

-- 地址：数据 (bit6-5:颜色, bit4-3:形状, bit2-1:角度, bit0:需放置标志)
-- 颜色编码: 00=红 01=蓝 10=黄 11=黑
-- 形状编码: 00=三角 01=六边 10=圆形 11=方形



0 : 0000000; -- [空位置]
1 : 0000000; -- [空位置]
2 : 0000000; -- [空位置]
3 : 0000000; -- [空位置]

4 : 0000000; -- [空位置]
5 : 0000000; -- [空位置]
6 : 0000000; -- [空位置]
7 : 0000000; -- [空位置]

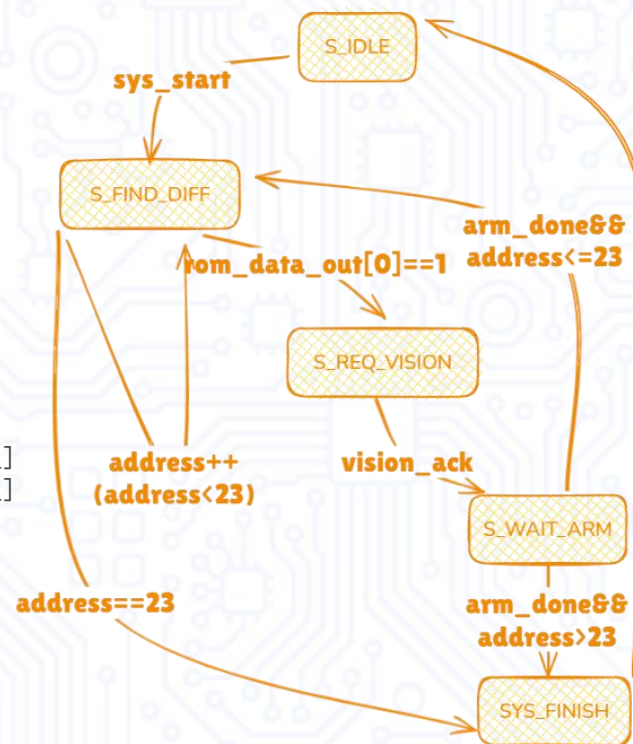
8 : 0011001; -- [空位置]
9 : 0010001; -- [空位置]
10 : 0001001; -- [空位置]
11 : 0000001; -- [空位置]

12 : 0111001; -- [空位置]
13 : 0110001; -- [空位置]
14 : 0101001; -- [空位置]
15 : 0100001; -- [空位置]

16 : 1111001; -- 蓝色 (01) 三角 (00) 角度00 需放置 (1)
17 : 1110001; -- 蓝色 (01) 六边 (01) 角度01 需放置 (1)
18 : 1101001; -- [空位置]
19 : 1100001; -- [空位置]

20 : 1011001; -- 红色 (00) 方形 (11) 角度00 需放置 (1)
21 : 1010001; -- 红色 (00) 圆形 (10) 角度00 需放置 (1)
22 : 1001001; -- 红色 (00) 六边 (01) 角度00 需放置 (1)
23 : 1000001; -- 红色 (00) 三角 (00) 角度00 需放置 (1)

END;



2.4.2 跨时钟域处理

图像处理和机械臂运动沟通信号的跨时钟域处理，由于图像处理模块使用的时钟是外部ov5640的pc1k端口的时钟，机械臂运动的时钟是系统时钟，二者进行通信时候需要进行跨时钟处理，做法是将图像处理产生的坐标有效信号延长7个图像处理时钟，机械单次运动完场的信号同样延长7个系统时钟周期，从而保证整个系统工作时得稳定性。

图像处理模块发送7周期的应答信号：

机械臂处理模块发出7周期的单次运动结束信号：

```
ACK_MATCH: begin
    stab timer <= 0;
    // 遍历目标寻找匹配项
    for (i = 0; i < 4; i=i+1) begin
        if(target_valid[i] && (shape_in[i] == target_shape)) begin
            temp_x <= center_x[i];
            temp_y <= center_y[i];
            vision_ack <= 1'b1;
            valid <= 1'b1;      // 立即标记有效
            ack_state <= ACK_SEND;
        end
    end
end
// 应答发送状态
ACK_SEND: begin
    if(hold_cnt<7)
        begin
            hold_cnt<=hold_cnt+1'b1;
        end
    else ack_state <= ACK_IDLE;
```

```
end
DONE_HOLD: begin
    if(hold_cnt<3'd4)
        hold_cnt<=hold_cnt+1'b1;
    else state <= IDLE;
end
//state <= IDLE;
default: state <= IDLE;
```



02 关键技术分析

Key Technology Analysis

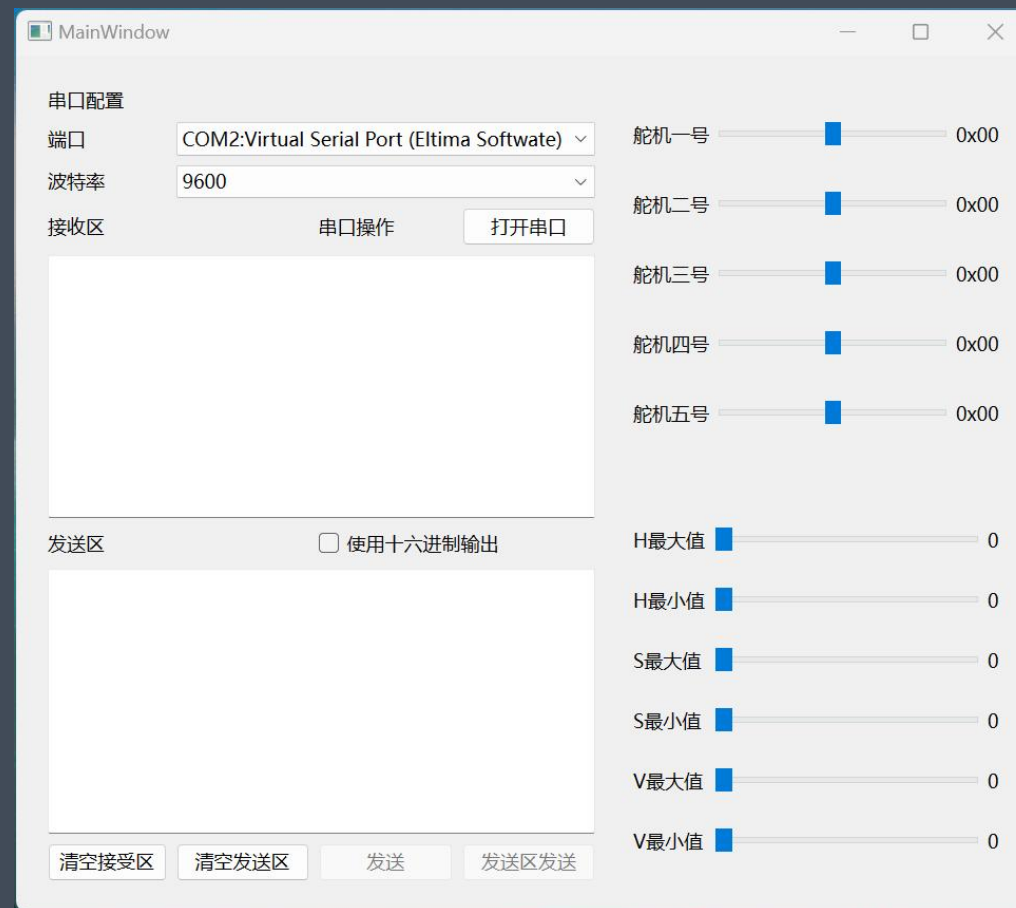


2.5 上位机设计与控制

2.5.1 软件架构设计

本软件基于 Qt 框架开发，主要用于对机械臂进行控制测试和hsv调参，支持三种工作模式切换，具备滑块参数调节、十六进制 / 十进制数据显示、串口通信配置及数据收发等功能。

上位机UI界面如图：

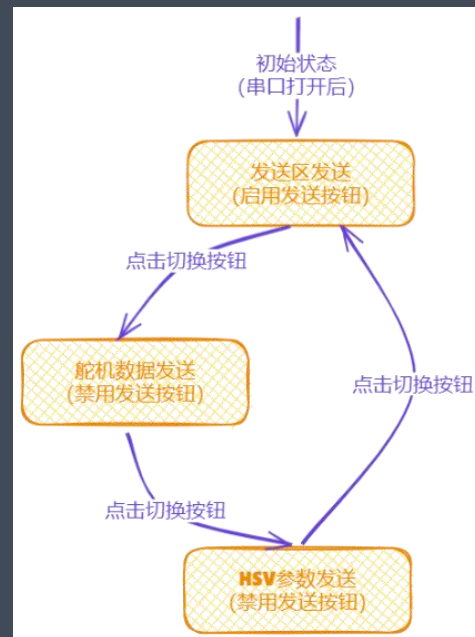


2.5.2 核心功能实现

串口通信：支持自动扫描端口、标准波特率选择，固定配置为8N1（8数据位、无校验、1停止位），打开串口后启用模式切换。

参数调节：前5滑块（127~127）控制舵机（负数转补码，显示十六进制），后6滑块（0~255）控制HSV（直接转无符号字节，显示十进制）。

模式切换：三种模式循环切换（发送区/舵机/HSV），对应激活不同组件并禁用其他组件。模式切换流程：





2.5.3 关键技术实现

十六进制处理：支持用户通过发送区输入十六进制字符串（如00 FF A0），需转换为字节数组发送；同时，滑块值生成的指令也需按十六进制格式封装。输入字符串去空格、校验长度（偶数）、逐字节转换，错误时提示格式异常。代码实现：

```
QByteArray MainWindow::hexStringToByteArray(const QString &hexStr) {  
    QString cleanedStr = hexStr.trimmed().remove(QRegularExpression("\\s+"));  
    if (cleanedStr.length() % 2 != 0) { /* 长度校验 */ }  
    for (int i = 0; i < cleanedStr.length(); i += 2) {  
        QString byteStr = cleanedStr.mid(i, 2);  
        char byte = static_cast<char>(byteStr.toInt(&ok, 16)); /* 逐字节转换 */  
        data.append(byte);  
    }  
    return data;  
}
```





模式状态机管理：通过枚举类型和状态更新函数实现三种模式的循环切换，控制 UI 控件的可用状态，避免用户误操作。代码实现：

```
void MainWindow::on_switchModeBtn_released() {  
    currentMode = static_cast<Mode>((static_cast<int>(currentMode) + 1) % 3);  
    updateModeUI();  
}  
  
void MainWindow::updateModeUI() {  
    switch (currentMode) {  
        case Mode::Mode1: ui->openPortBtn_2->setText("发送区发送"); ui->sendBtn->setEnabled(true); break;  
        case Mode::Mode2: ui->openPortBtn_2->setText("舵机数据发送"); ui->sendBtn->setEnabled(false); break;  
        case Mode::Mode3: ui->openPortBtn_2->setText("HSV参数发送"); ui->sendBtn->setEnabled(false); break;  
    }  
}
```





2.5.4 错误处理与异常机制

串口错误：打开失败时提示端口名称及错误信息（如权限不足、端口占用），波特率不匹配需手动重新配置。 数据格式错误：十六进制长度奇数或包含无效字符时弹出警告，终止发送操作。

2.5.5 使用指南

快速操作：

连接设备→选择端口/波特率→打开串口→切换模式→调节参数或输入数据。

模式要点：

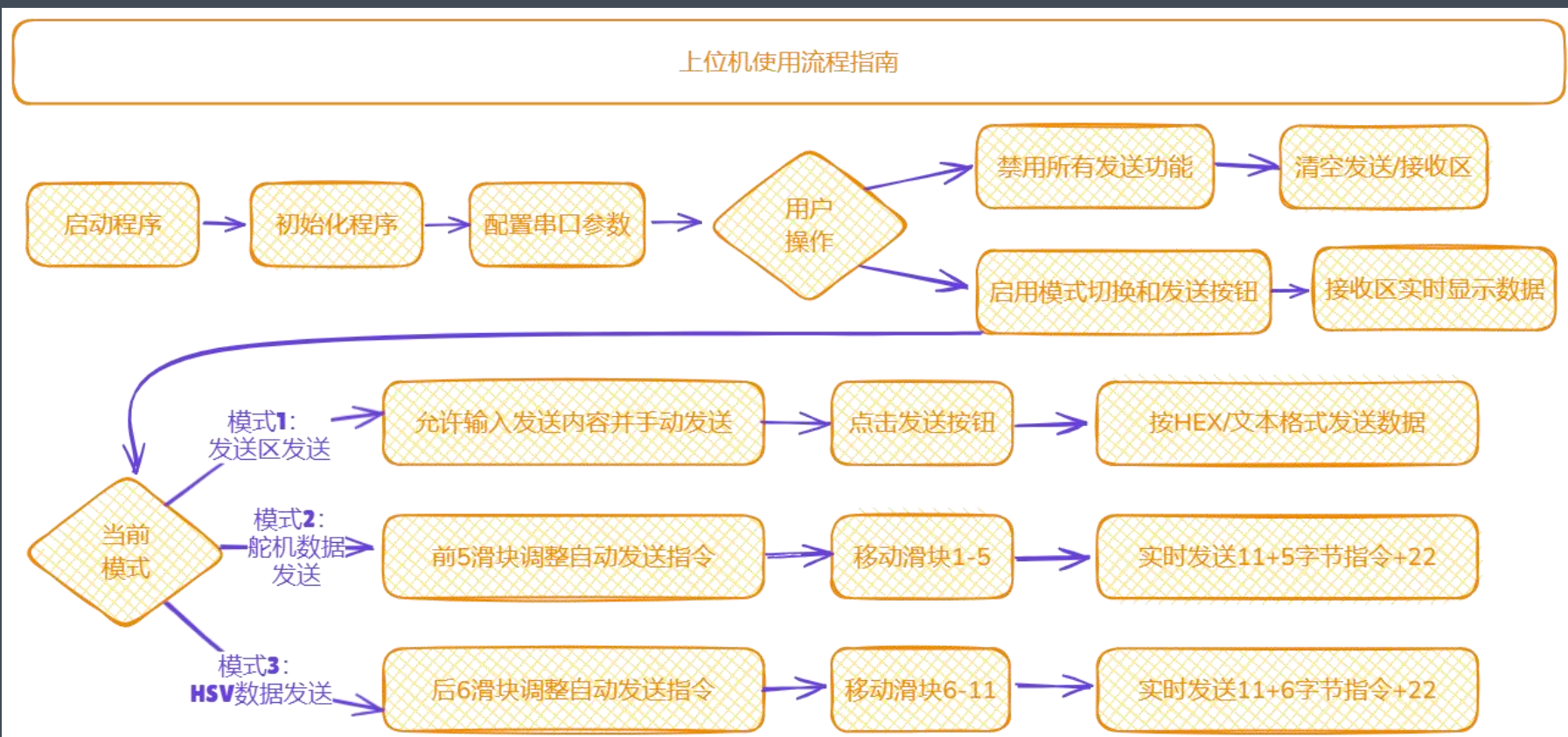
发送区模式：支持ASCII/十六进制，勾选复选框切换格式。

舵机模式：滑块值对应舵机角度（127~127），标签显示十六进制值。

HSV模式：滑块值直接映射为0~255的无符号数，标签显示十进制值。

操作流程如图：







03

性能指标

Performance Indicators

3.1 目标锁定 时间

3.1.1. 图像采集与预处理耗时分析

系统首先接收RGB888格式的原始图像（640x480@60fps），通过 RGB2HSV模块 在11个时钟周期（148.2ns）内完成色彩空间转换，提取色调（H）、饱和度（S）、明度（V）分量。随后 HSV二值化模块 在2周期（26.94ns）内根据目标颜色阈值生成二值图像（如黄色物体对应H=50-80，S>135）。

3.1 目标锁定 时间

3.1.2. 形态学处理耗时分析

二值图像先后经过 腐蚀（6周期/80.82ns）和 膨胀（6周期/80.82ns）处理：

腐蚀 使用3x3核消除细小噪点，通过移位寄存器和两级与操作实现；
膨胀 同样采用3x3核恢复目标尺寸，通过两级或操作完成。

此阶段共消耗12周期（161.64ns），有效分离粘连目标并平滑边缘。

3.1 目标锁定 时间

3.1.3. 目标检测与形状识别耗时分析

shape_recognition模块 启动三帧检测流程：

首帧（N帧）：检测目标初始位置，记录边界框

（left/right/top/bottom）；

次帧（N+1帧）：跟踪目标运动轨迹，更新边界坐标；

末帧（N+2帧）：计算垂直投影面积（通过4个专用BRAM存储累加值），根据面积阈值分类形状（正方形>4000像素，圆形>3100像

3.1 目标锁定 时间

3.1.4. 坐标转换与输出耗时分析

中心计算：取边界框均值获得像素坐标（如(320, 240)）；

X坐标： $(x-320) \times 0.46$ （Q16定点数乘法后截断为整数mm）

Y坐标： $(y-240) \times 0.42 + 172$ （含机械坐标系偏移补偿

应答握手：通过valid和vision_ack信号同步外部设备。

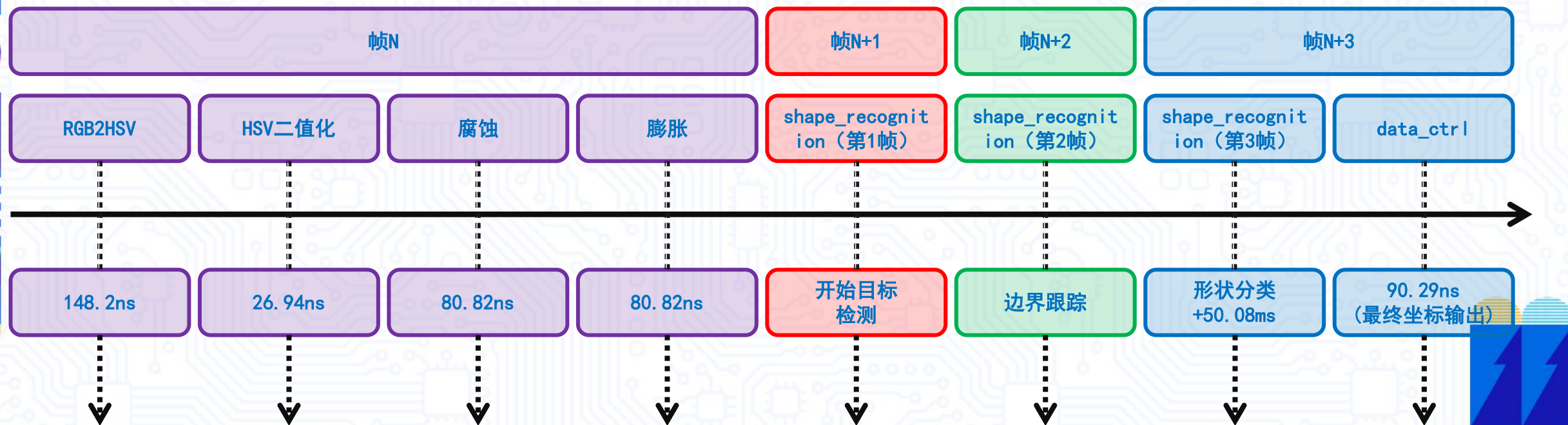
03 性能指标

Performance Indicators



3.1.5 整体分析耗时分析

Img_pro模块的时序如下



03 性能指标

Performance Indicators



各模块时间消耗如下

总结：从物块进入视野到输出稳定坐标需要约50.1毫秒

处理阶段	延迟 (T: 周期)	关键说明
RGB转HSV (RGB2HSV)	11T (148.2ns)	含除法器流水线
HSV二值化 (HSV_binsrixstion)	2T (26.94ns)	并行阈值比较
腐蚀处理 (Bit_Erosion_Detector)	6T (80.82ns)	3x3核运算+行缓存
膨胀处理 (Bit_Dilation_Detector)	6T (80.82ns)	与腐蚀对称设计
形状识别 (shape_recognition)	3帧+4T	三帧检测机制+实时计算 (约50.08ms)
坐标转换 (data_ctrl)	7T (94.29ns)	Q16定点数运算
总和 (流水线化)	≈50.1ms	受限于shape_recognition的三帧检测



3.2 准确度

3.2 吸取准确度与仓库放置准确度

再设置好各中误差补偿参数后后，我们的机械臂吸取准确度可以达到百分之90左右，仓库放置准确度再百分之70左右

3.3 移动速度

3.3 移动速度

控制移动速度的方法我们采用的是使用计数器计数，当计数到一定数值时候，让控制舵机的pwm波的占空比步进1，由此达到控制速度的效果。

03 性能指标

Performance Indicators



代码实现如下

```
// 运动控制
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        speed_cnt <= 0;
        current_angle <= INIT_ANGLE;
    end
    else if (en && !angle_error) begin // 添加en使能控制
        if (current_angle == target_angle) begin
            speed_cnt <= 0;
        end
        else begin
            if (speed_cnt == INTERNAL_SPEED) begin
                speed_cnt <= 0;
                current_angle <= (current_angle < target_angle) ?
                    current_angle + 1 : current_angle - 1;
            end
            else begin
                speed_cnt <= speed_cnt + 1;
            end
        end
    end
end
```

经测试在各个舵机速度设置为如下是机械臂工作速度较快且运动过程不出磕碰

```
parameter INTERNAL_SPEED = 200_000
parameter SPEED0 = INTERNAL_SPEED/3;
parameter SPEED1 = INTERNAL_SPEED/3;
parameter SPEED2 = INTERNAL_SPEED/3;
parameter SPEED3 = INTERNAL_SPEED/3;
parameter SPEED4 = INTERNAL_SPEED/4;
```

加上视觉处理时间和运动过程中状态的稳定时间（仓库稳定时间2s，目标上方稳定时间2s），处理十六个物块需要的时间在4分30秒左右。



04

可优化方向

Optimizable Orientation

04 可优化方向

Optimizable Orientation

系统架构优化

优化方案：

- 引入高级建模工具：使用Simulink等高级建模工具进行系统级仿真与算法开发。这有助于缩短开发周期，提高产品性能，并通过图形化界面快速构建复杂的运动规划模型。
- 设计高速数据采集接口：针对传感器反馈的数据量大且频率高的特点，设计高速数据采集接口，确保数据的实时性和准确性。

执行细节：

- Simulink集成：在FPGA开发环境中集成Simulink，利用其丰富的库函数和图形化界面进行系统建模和仿真。
- 接口电路设计：设计高速ADC/DAC模块连接外部设备，确保传感器数据的快速采集和传输。

04 可优化方向

Optimizable Orientation

机械臂控制模块优化

优化方案：

- 引入误差补偿算法：通过逆向补偿来弥补机械臂的结构性误差，提高控制精度。
- 优化逆运动学解算：使用查找表结合实时计算的方式，优化逆运动学解算过程，提高计算效率。
- 改进PWM信号发生器：优化PWM信号发生器的设计，提高机械臂的运动速度和稳定性。

执行细节：

- 误差补偿实现：对机械臂进行运动学和动力学建模，找到每个关节、每个执行单元的误差源。通过计算给出一个反向调整值，实现误差补偿。
- 逆运动学解算优化：预先计算并存储一组关节转动角度，形成查找表。在实际控制中，根据目标位置快速查表得到近似角度，再进行实时计算修正。
- PWM信号发生器设计：使用Verilog等硬件描述语言编写高效的PWM信号发生器，确保信号的稳定性和准确性。

04 可优化方向

Optimizable Orientation

任务控制模块优化

优化方案：

- 改进跨时钟域处理机制：确保图像处理模块和机械臂控制模块的稳定通信。
- 优化任务调度算法：提高多任务处理能力和系统响应速度。

执行细节：

- 跨时钟域处理：采用异步FIFO或双口RAM等机制实现跨时钟域数据传输，确保数据的完整性和稳定性。
- 任务调度算法：设计优先级调度算法，根据任务的重要性和紧急程度进行任务调度。同时，优化任务队列管理，减少任务切换开销。

04 可优化方向

Optimizable Orientation

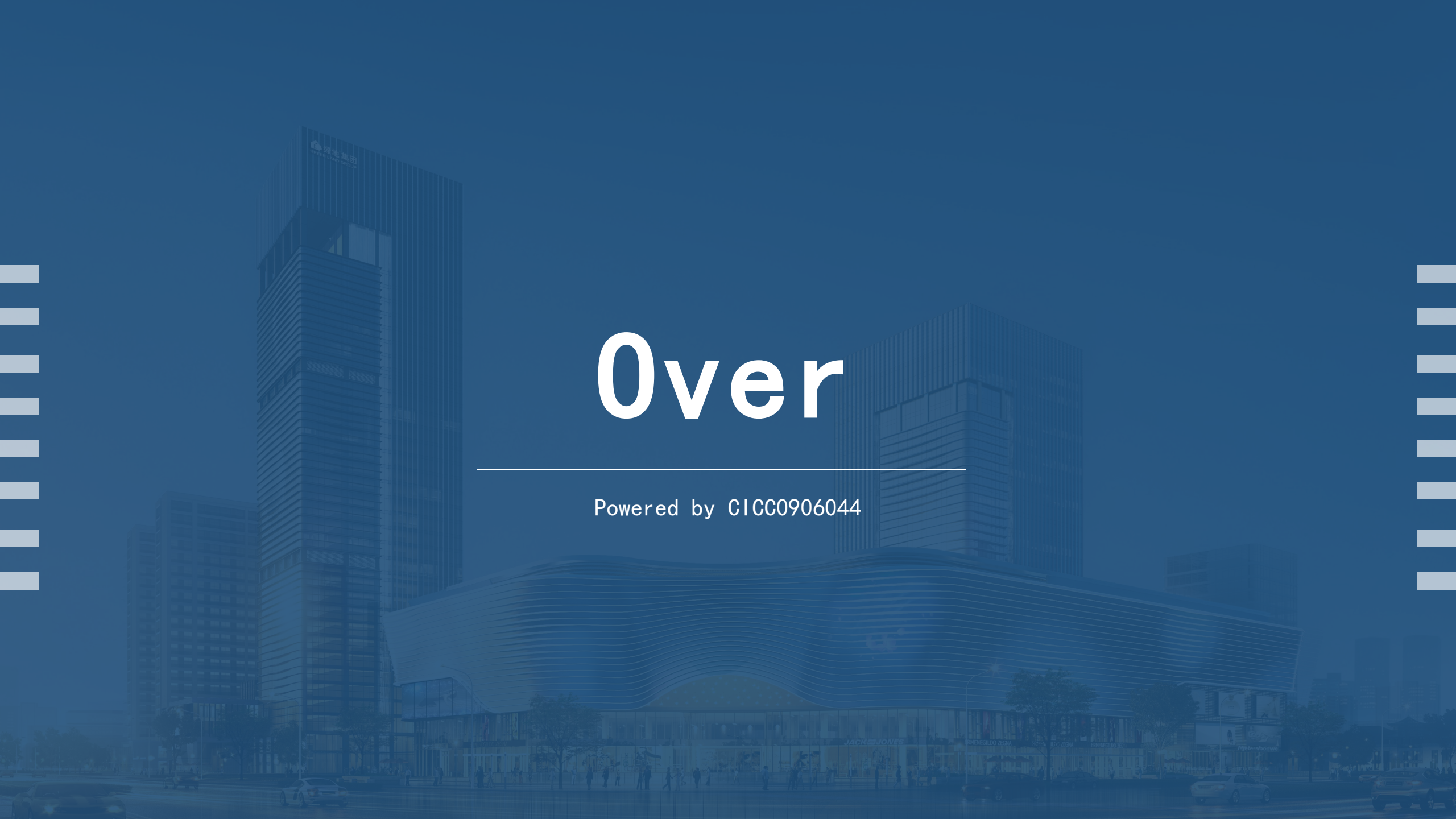
系统稳定性提升

优化方案：

- 设计高性能的驱动电路：确保足够的电流和电压承受能力，以驱动机械臂的执行器。
- 开发碰撞检测功能和故障诊断与保护功能：提高系统的安全性和稳定性。

执行细节：

- 驱动电路设计：设计高性能的驱动电路，采用适当的功率放大器和保护电路，确保机械臂执行器的稳定驱动。
- 碰撞检测与故障诊断：通过实时检测机械臂的位置和速度，判断是否存在碰撞风险。同时，对机械臂的各个部件进行实时监测，发现故障及时进行诊断和保护。



Over

Powered by CICC0906044